

Classifying IBM Watsonx Documentation Pages by User Intent

5LN712 Information Retrieval — Assignment 1

1. Introduction

Technical documentation is heterogeneous in *purpose*: some pages explain a concept, some enumerate parameters, others walk a user through a procedure. A retrieval system that knows the *type* of a page can route a query — “*how do I deploy a model?*” versus “*what is a deployment space?*” — to the right kind of result. This work builds a small, document-level binary classifier over real IBM Watsonx documentation pages, using sentence-transformer embeddings with linear classifiers. The task is best understood as *genre* or intent classification rather than topical classification (Finn & Kushmerick, 2003).

2. Dataset

The source corpus is `ibm-research/watsonxDocsQA` (originally 1,144 English documentation pages). The classification is at page level – each page is labelled by the type of information a user would primarily use it for. Model input is constructed as `title + first 800 words of body` to preserve the title signal while constraining length.

Label schema. The initial plan used four classes: `task`, `concept`, `troubleshooting`, `reference`. During inspection, this was simplified to a binary schema: `conceptual` (`concept + reference`) versus `how-to` (`task + troubleshooting`). Two reasons motivated the change. First, `troubleshooting` was rare in the corpus and would have introduced a small standalone class at this dataset size. Second, `concept` and `reference` could not be separated consistently, since both serve a *read-to-understand* intent rather than a *read-to-act* one.

Annotation process. First, a heuristic script generated candidate labels based on title and body. The auto-generated labels were also marked with confidence levels (`high / medium / low`). Then most candidates were manually reviewed except for the `how-to + medium confidence` combinations (333 rows), given that sufficient verified samples for this assignment had been obtained. A further 17 rows were dropped, because the source text had not been scraped correctly or was too ambiguous to label. From the remaining 794 cleanly labelled pages (563 `conceptual` / 231 `how-to`), a balanced subset of 400 was sampled and split 70/15/15, yielding 280 train / 60 validation / 60 test, with both classes equally represented in every split.

3. Methods

Each page is encoded by a pre-trained sentence-transformer used as a frozen feature extractor, and the resulting 384-dimensional embedding is fed to a shallow linear scikit-learn classifier (Logistic Regression or LinearSVC). The experiments did not implement any fine-tuning. This setup follows Sentence-BERT, which shows that fixed-length embeddings from such models pair well with linear classifiers on short texts (Reimers & Gurevych, 2019).

Three conditions were run, each isolating one design variable against the baseline:

ID	Embedding	Classifier
A (baseline)	all-MiniLM-L6-v2	LogisticRegression
B	all-MiniLM-L6-v2	LinearSVC
C	bge-small-en-v1.5	LogisticRegression

Condition C uses the BGE family, which is competitive on English MTEB tasks (Xiao et al., 2023). Hyperparameters are default: LogisticRegression `C=1.0`, `max_iter=1000`; LinearSVC `C=1.0`, `max_iter=2000`; both with seed 42. The validation split was used only for sanity checks; final reported metrics are on train and test as required.

4. Results

Condition	Train Acc	Train F1	Test Acc	Test F1
A	0.879	0.879	0.817	0.817
B (Best)	0.971	0.971	0.867	0.867
C	0.864	0.864	0.833	0.833

Condition B (MiniLM + LinearSVC) achieves the best test macro F1 (0.867) and is the model deployed in the Hugging Face demo. The interesting comparison is between the two ablations against the baseline: switching the classifier from LogisticRegression to LinearSVC under the same MiniLM embedding (A → B) gives a +5.0 F1-point improvement, while upgrading the embedding but keeping LogisticRegression (A → C) gives only +1.6 F1 points. In other words, the classifier choice mattered more than the embedding choice on this dataset — consistent with the long-standing observation that linear SVMs fit high-dimensional, near-linearly-separable text features particularly well (Joachims, 1998). One caveat is that B also shows the largest train–test gap (0.971 vs. 0.867), which suggests some overfitting. Stronger regularisation or a larger dataset would likely close part of this gap.

Error analysis. Two recurring patterns were found in the misclassified cases. The most common errors happen on pages whose titles open with action gerunds — *Creating...*, *Defining...*, *Specifying...* — while their short descriptions use “*you can*” phrasing to show what users can do with the tool. These pages are mostly conceptual, but the pattern is a stronger signal for tasks, and the classifier struggles to read the subtle difference from this surface form alone. The second failure occurs on long procedural pages, which typically open with informational context or related concepts before the actual ordered steps. Because the model input is truncated to the first 800 words, the procedural body can fall outside the window, leaving the classifier to decide on what is effectively a conceptual preamble and mislabel the page as conceptual. Both errors line up with the borderline cases noted during annotation, and the second one points to a clear fix in the input setup: a longer or section-aware cutoff would likely recover many of these cases.

5. Discussion and Limitations

The reported numbers come with several caveats. On the data side, the dataset is small (400 examples from a single product’s documentation), all annotation was performed by a single annotator, and no inter-annotator agreement could be measured — a second annotator would be needed before the labels could be treated as a stable evaluation signal. On the task side, the class boundary itself is blurry. Even under the original four-class schema, troubleshooting pages often look like reference pages but are really meant to help users complete a task. Collapsing the schema into two classes does not erase this overlap — it just folds it into the two new labels. The corpus is also stylistically uneven: it has been written and maintained by many authors over a long period, so the linguistic signal for each class is weaker than what you would see in a benchmark built for this purpose.

Within these limits, the 80–87% test F1 range is a reasonable upper bound for what a frozen-embedding + shallow-classifier setup can extract from this signal at this scale. Substantial gains would likely require some combination of more data, fine-tuned encoders, features beyond the first 800 words (e.g., section headings or code-block density), and a cleaner annotation protocol with multiple annotators.

6. Reflection on AI Tool Use

I treated this project like running a small development team. I stayed in the role of project manager — keeping the full picture of the project and making the final calls — while several AI assistants took on more focused roles: system engineer, senior programmer, and technical writer. Each assistant only saw the part of the project that was relevant to its task, and I curated and passed on their outputs as the work moved forward.

Within this setup, the assistants worked well as consultants and were especially good at writing code. They are also great brainstorming partners, and they help keep scattered ideas organised. They handle the details well once the frame is set, but the judgement calls — and the final decisions — still have to come from a human after taking a closer look. A second, more practical limit was context management: a real share of the project’s time went into preparing handoff documents, because each role only had a limited context window to work with.

The broader pattern is clear: AI is great at generating ideas and exploring options, but getting the work done in an orderly way still needs a human to set the process.

Appendix: Project Links

- GitHub repository: <https://github.com/jennyhuang17/watsonx-docs-type-classifier>
- Hugging Face dataset: <https://huggingface.co/datasets/itsjhuang/watsonx-docs-document-type>
- Hugging Face demo: <https://huggingface.co/spaces/itsjhuang/watsonx-docs-type-classifier>

References

- Finn, A., & Kushmerick, N. (2003). Learning to classify documents according to genre. *IJCAI Workshop on Information Retrieval and the Web*, 7(1), 1–10.
- Joachims, T. (1998). Text categorization with Support Vector Machines: Learning with many relevant features. *Machine Learning: ECML-98*, 137–142.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP 2019*, 3982–3992.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.